

Modeling and Design Techniques

Presented by: Dr. Abdolraheem Khader
May 2025

Introduction to Modeling and Design Techniques

- **Definition:** Methods used to represent, visualize, and design software and database systems.
- **Objective:** Simplify complex systems through visual representation.
- **Importance:** Enhances understanding, communication, and system documentation.

Part 1: Unified Modeling Language (UML)

- **Definition:** A standardized modeling language for specifying, visualizing, and documenting software systems.
- **Purpose:** Represent software architecture and interactions between components.
- **Common UML Diagrams:**
 - Use Case Diagram.
 - Class Diagram.
 - Sequence Diagram.
 - Activity Diagram.

Part 1: Unified Modeling Language (UML)

- **Benefits:**
 - Standardized communication.
 - Clear system visualization.
 - Effective documentation.
 - Supports object-oriented (OO) analysis and design.
- **Applications:** Software design, system architecture, process modeling.

Introduction to Use Case Diagrams

- **Purpose:** Visualize the functional requirements of a system.
- **Components:**
 - Actors: External entities interacting with the system.
 - Use Cases: Functionalities provided by the system
 - Relationships: Associations between actors and use cases
- **Example:** Online shopping system.

Elements of Use Case Diagrams

- **Actor:** Represents a user or system interacting with the application.
- **Use Case:** A functionality or service provided.
- **Association:** Link between an actor and a use case.
- **Include/Extend:** Relationships for use case decomposition and variation
- **Example:** User logging in to an application

Use Case Diagram Example

- **Scenario:** Library Management System.
- **Actors:** Student, Librarian.
- **Use Cases:** Borrow Book, Return Book, Search Catalog.
- **Relationship:** Student -> Borrow Book.

Introduction to Class Diagrams

- **Purpose:** Model the static structure of a system by showing its classes, attributes, and relationships.
- **Components:**
 - Classes.
 - Attributes.
 - Methods.
 - Relationships (Association, Aggregation, Inheritance).

Elements of Class Diagrams

- **Class:** Blueprint of an object (name, attributes, methods).
- **Association:** Simple connection between classes.
- **Aggregation:** A "whole-part" relationship.
- **Inheritance:** Generalization between classes.
- **Example:** A "Vehicle" superclass with "Car" and "Truck" subclasses.

Class Diagram Example

- **Scenario:** E-Commerce System.
- **Classes:** User, Product, Order.
- **Attributes:** User (name, email), Product (ID, price), Order (order number).
- **Relationships:**
 - User places Order.
 - Order contains Products.

Introduction to Sequence Diagrams

- **Purpose:** Model the flow of messages between objects in a system.
- **Components:** .
 - Lifeline: Represents an object's existence over time
 - Activation: Shows the time an object is active
 - Message: Interaction between objects
 - Return Message: Response from an object
- **Example:** User logging in to an application.

Elements of Sequence Diagrams

- **Actors:** Initiate interactions.
- **Objects:** System components involved in communication.
- **Messages:** Arrows indicating the flow of information.
- **Lifelines:** Vertical lines indicating object existence.
- **Example:** Customer ordering a product online.

Sequence Diagram Example

- **Scenario:** ATM Transaction.
- **Steps:**
 - User inserts card.
 - System validates card.
 - User enters PIN.
 - System verifies and displays options.
- **Visual:** ATM system interactions.

Introduction to Activity Diagrams

- **Purpose:** Model the dynamic aspects and workflows within a system.
- **Components:**
 - Activities: Actions or tasks.
 - Decision Nodes: Conditional branching.
 - Start/End Points: Indicating process boundaries.
- **Example:** Order processing in an e-commerce system.

Elements of Activity Diagrams

- **Start/End:** Indicate the process's beginning and end.
- **Activity:** Represents a specific task.
- **Decision:** Forks based on conditions.
- **Merge:** Combines multiple branches.
- **Example:** User registration process.

Activity Diagram Example

- **Scenario:** Online Payment Processing.
- **Activities:** Enter card details -> Validate -> Process Payment -> Confirm.
- **Decision Point:** Validating card information.

Part 2: Entity-Relationship Diagrams (ERD)

- **Definition:** Visual representation of entities and their relationships in a database.
- **Purpose:** Database structure modeling.
- **Components:**
 - Entity: Represents a data object.
 - Attribute: Characteristic of an entity.
 - Relationship: Association between entities.

ERD Symbols and Notations

- **Entity:** Rectangles.
- **Attribute:** Ovals.
- **Relationship:** Diamonds.
- **Primary Key:** Underlined attribute.
- **Foreign Key:** Dashed line to related entity

Types of Relationships in ERD

- **One-to-One (1:1).**
- **One-to-Many (1:M).**
- **Many-to-Many (M:M).**
- **Example:** A customer can place multiple orders, but an order belongs to one customer.

ERD Example: Library System

- **Entities:** Book, Member, Loan.
- **Attributes:** Book (ISBN, title), Member (ID, name), Loan (date).
- **Relationships:**
 - Member borrows Book
 - Book has many Loans

UML vs. ERD: Key Differences

- **UML:** Focuses on software system design.
- **ERD:** Focuses on database structure.
- **Use Cases:** UML for software architecture, ERD for database modeling.
- **Example:** UML for user interaction modeling, ERD for data storage

Best Practices in Modeling

- **Clarity:** Use consistent naming and notation.
- **Simplicity:** Avoid overly complex diagrams.
- **Consistency:** Align models with real-world processes.
- **Validation:** Ensure accuracy through stakeholder feedback.

Real-World Case Study

- **Scenario:** Designing an online booking system.
- **UML:** Use case diagram for user interactions.
- **ERD:** Database schema for customer and booking details.

Challenges in Modeling

- **Ambiguity:** Misinterpretation of symbols.
- **Over-Complexity:** Hard-to-read diagrams.
- **Lack of Standardization:** Inconsistent use of elements.
- **Solution:** Follow standardized practices (e.g., UML 2.5)

Thank You!